

Model Tuning

So till now, we've learned how to build machine learning models — but there's one big truth about models:

The first version of your model is never the best version.

That's where Model Tuning comes in.

When we train a model, it has certain settings — these are called hyperparameters. These hyperparameters decide how your model behaves — like how deep should a tree be, how many neighbors in KNN, or what learning rate to use

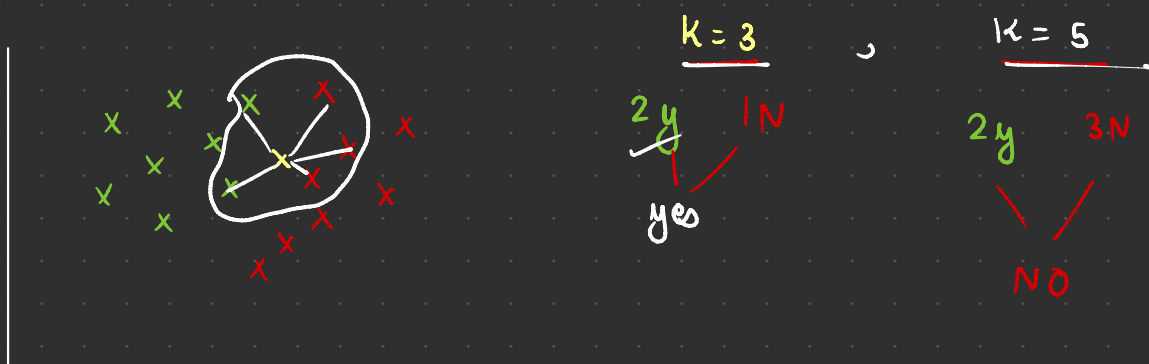
But if we just use random hyperparameters — our model might underfit, or overfit, or simply perform poorly.

So the goal of Model Tuning is:

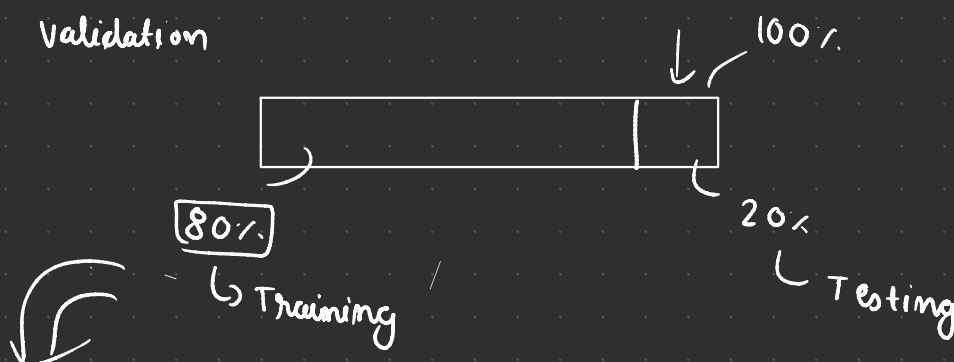
- 1) To find the best combination of hyperparameters
- 2) So that our model performs better on new, unseen data — not just on the training data.

In short — tuning helps you squeeze out the best possible performance from your model — and that's why it's an essential step before finalizing any ML model. And this is also called hyper parameter tuning.

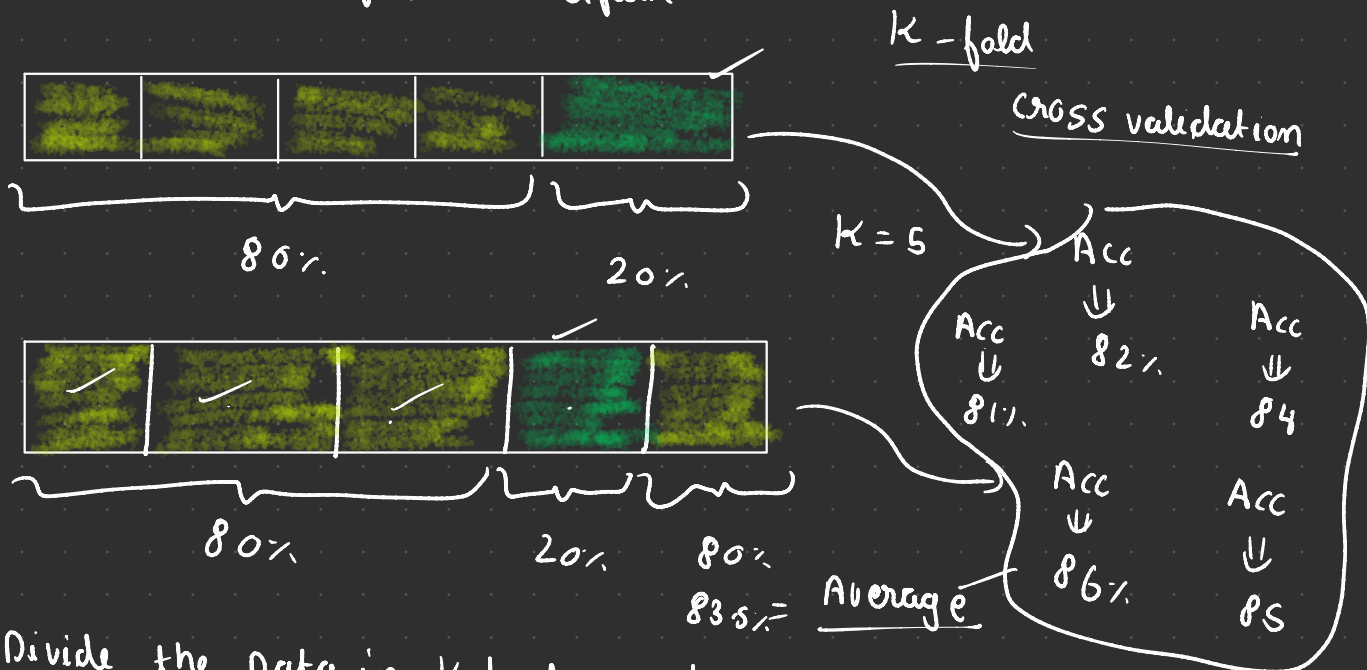
But before starting any thing we first have to see what is Cross Validation and how to use it.



Cross Validation



Overfit, Underfit, Poor Perform



You Divide the Data in K parts and every Part will be your testing Data Remaining will be your training Data

hyper Parameter tuning

Ridge & Lasso Regression = α = Learning Rate

12 NN $\Rightarrow$ n -neighbors = [3, 5, 7, 9]

Decision Tree

- ↳ max depth
- ↳ sample split
- max feature

SVM

- ↳ Kernel()

Random forest

DB Scans

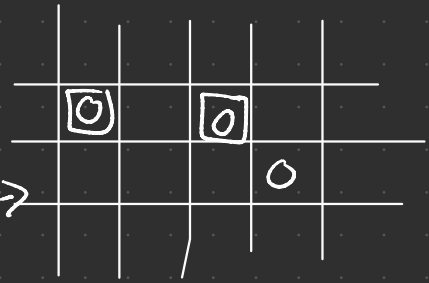
methods

- ① manual search
- ② Grid search CV
- ③ Randomized search

Grid Search CV

↙ ↘

Cross validation



1KNN

$N = [\underline{3}, \underline{5}, 7, 9, 11, \underline{13}]$

weights = ["~~uniform~~", "distance"]

metric = ["~~manhattan~~", "euclidean"]

X ↔ X

= { 9
distance
manhattan } Acc

Avg
Acc ← Experiment

↓

CV = 5

N	weight	metric	
3	uniform	manhattan	= Acc = ?
3	distance	manhattan	= Acc = ?
3	uniform	euclidean	= Acc = ?
3	distance	euclidean	

Random search CV

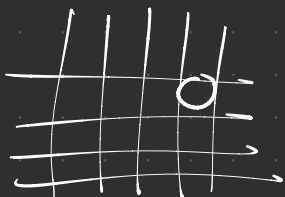
1KNN

N = 6

weight = 2

metric = 2

= $6 \times 2 \times 2 = 24$ ✓✓✓



(30)

X G Boost

Learning Rate = 10 = $10 \times 20 \times 10$

n_estimator = 20

maxDepth = 10

2000 ✓✓✓

Ensemble learning

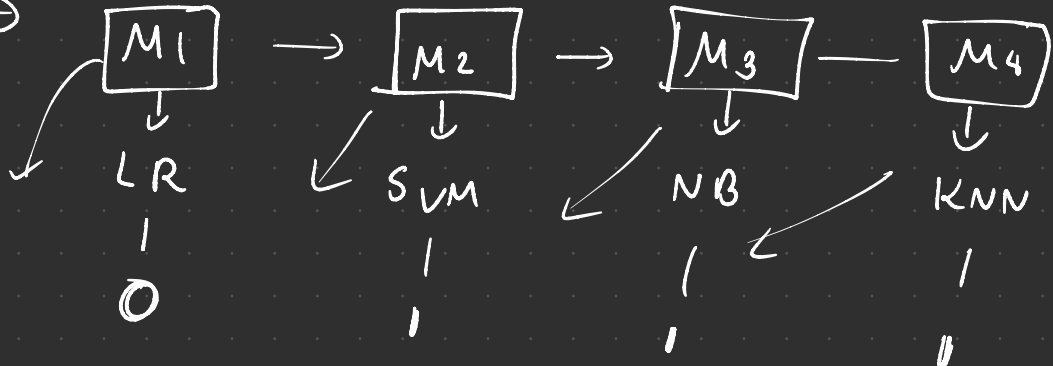
⚡ we hear the crowd ⚡

tying Speed	Internet Usage	hacker
26	12	0
58	36	1

Machine learning

Train

Test →



Query - TS = 5.6

90 = 28

hacker = ?

34 IN

Regression - mean instead of voting

Types of ensemble learning

Ensemble

Bagging

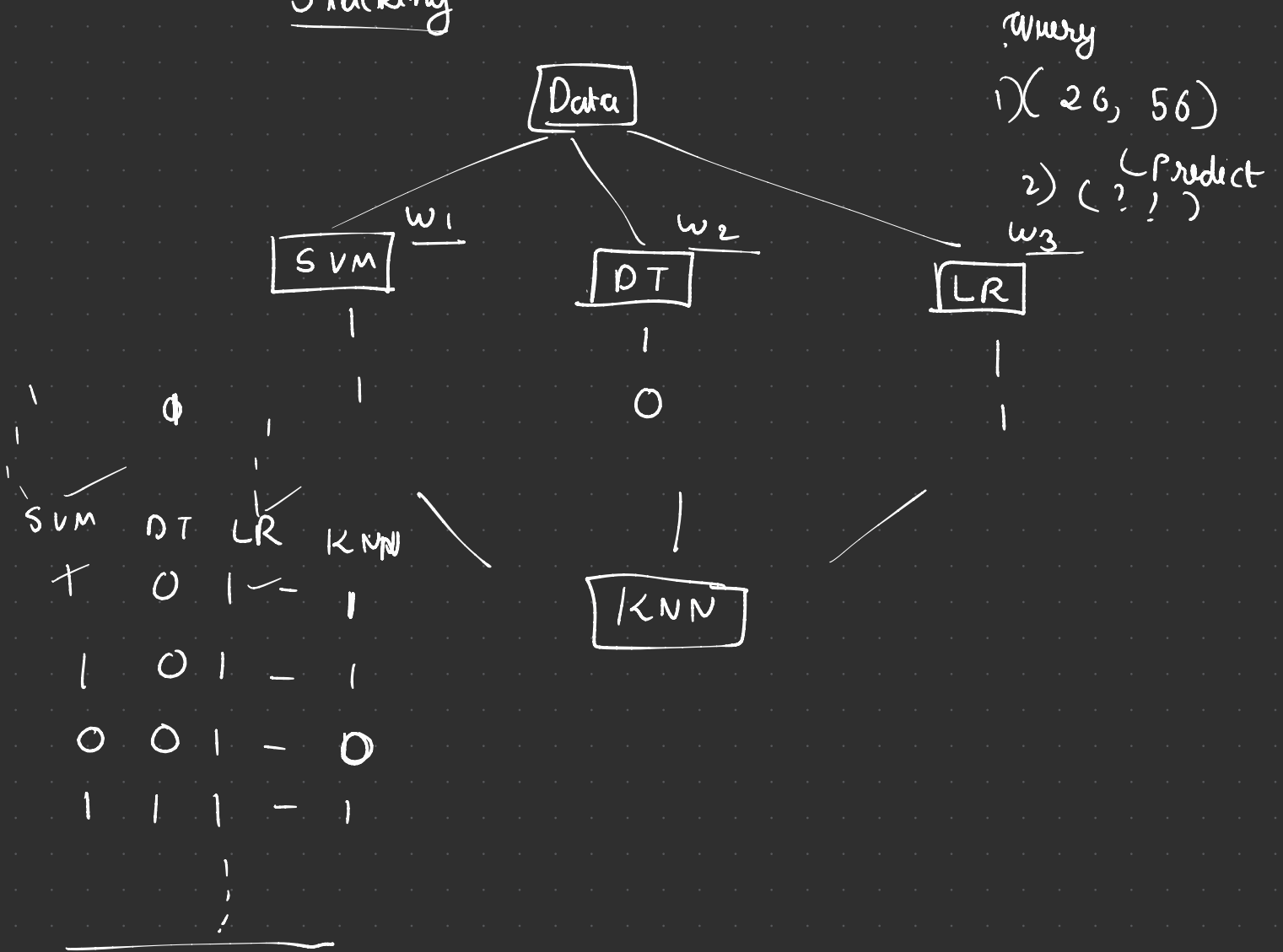
Boosting

Stacking

→ Random Forest ←

→ AdaBoost
→ Gradient Boost
→ XG Boost

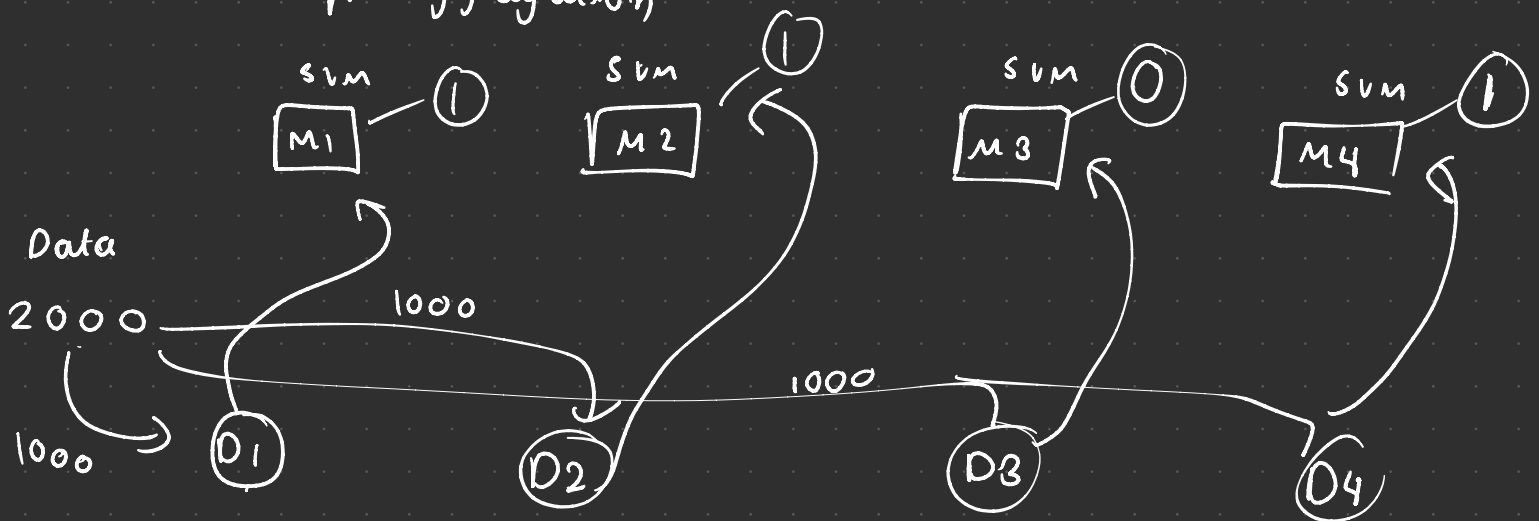
Stacking



Bagging

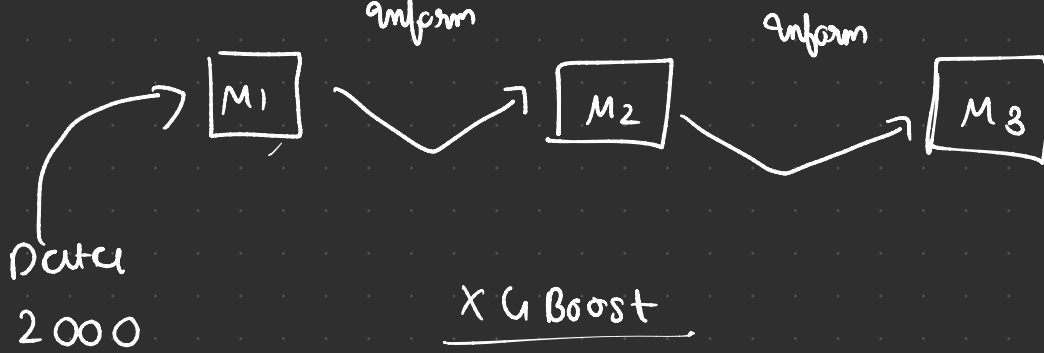
Bootstrap Aggregation

$w = 1$



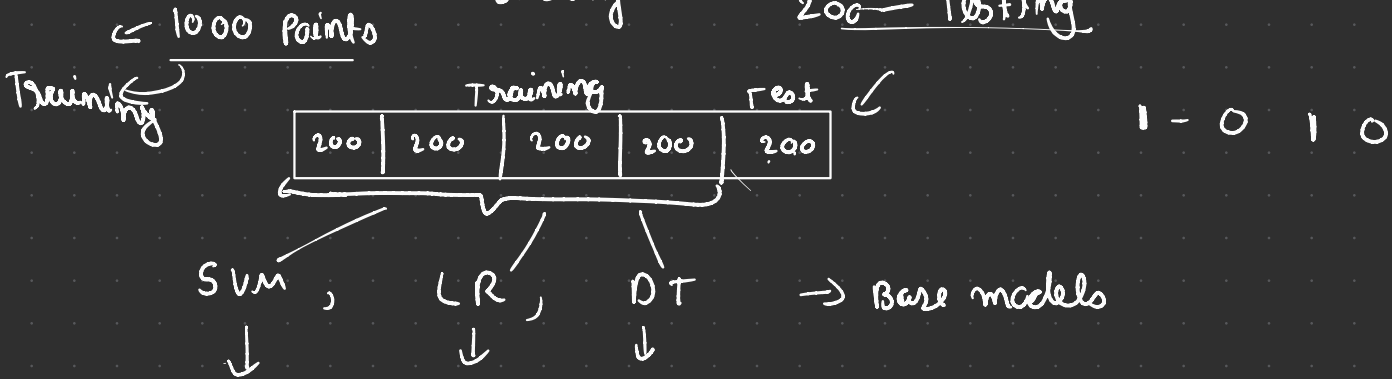
Random forest

Boosting



Stacking

200 — Testing



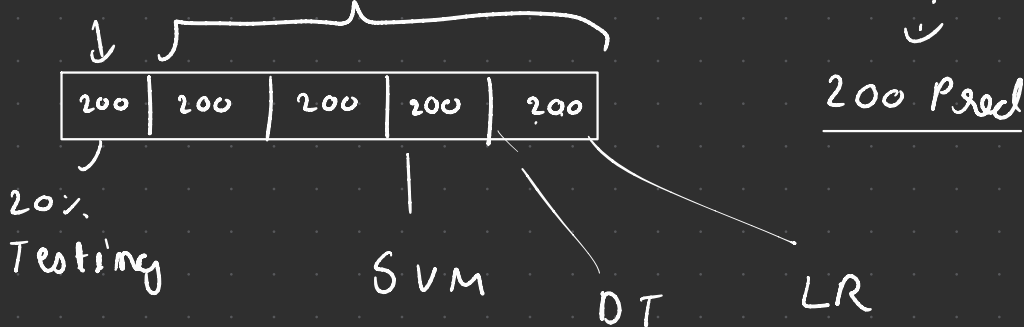
1 - 0 1 0

Meta model → KNN →

↓	↓	↓	↓
SVM_Pred	LR_Pred	DT_Pred	KNN
0	<u>1</u>	0	0
1	1	0	1
0	0	1	0

200 results

training



Meta model - 200 + 200 = 400

↳ 1000 — Pred

Bagging

LR, LR, LR 1000
800



Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

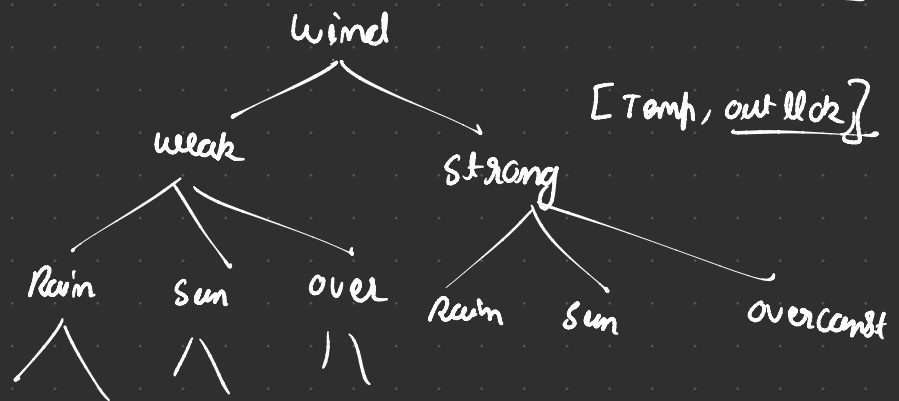
Random forest classifier

100 Decision Trees

q → play

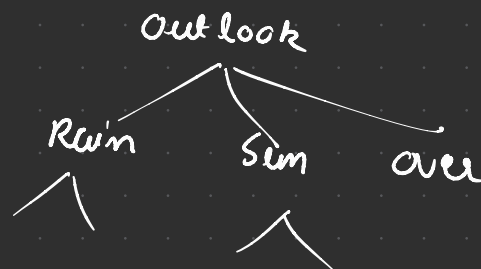
1 Tree - 600DP

→ Root Node [Temp, wind]

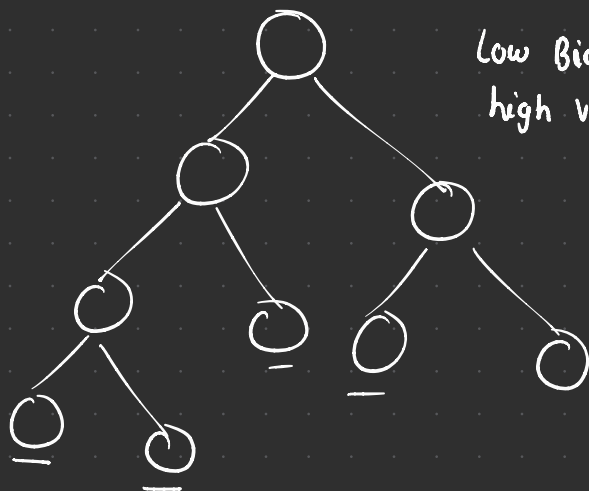


Tree - 2 - 600DP

[Outlook, Temp]



Bagging



Low Bias
high variance

Training
Test

Low Bias

Low Variance

overfitting

Generalized

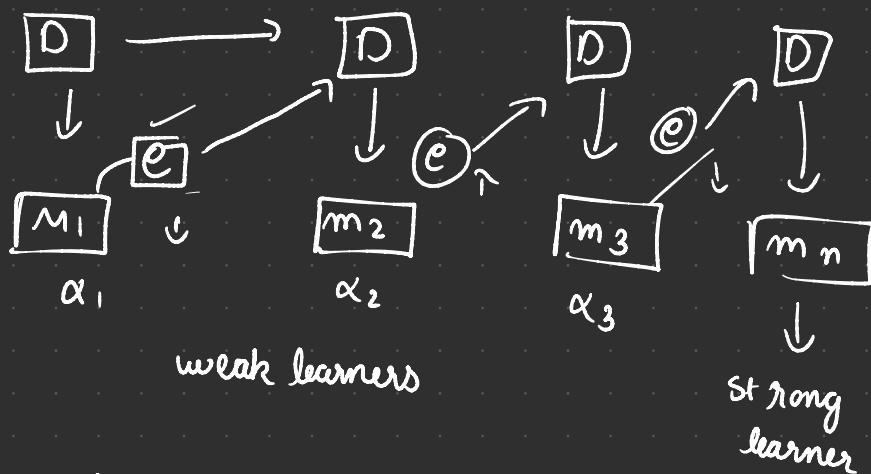
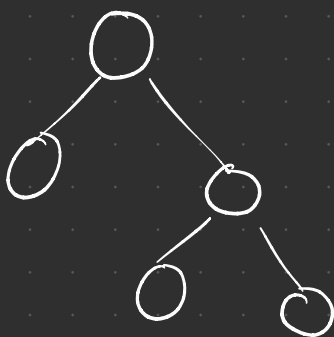
Boosting

underfitting

Generalized

High Bias, High Variance

Low Bias, Low Variance



$$f = \alpha_1(m_1) + \alpha_2(m_2) + \alpha_3(m_3) + \alpha_n(m_n)$$

① Ada boost

② Gradient Boosting

③ XG Boost

Unsupervised Learning

1) What is Unsupervised Learning?

Unsupervised Learning is a type of Machine Learning where:

The data does NOT have labels — no "right answer" is given.

The model must find patterns, structure, and relationships in the data by itself.

2) Goal of Unsupervised Learning:

Find hidden structure in the data:

Group similar data points (Clustering)

Reduce data dimensions while keeping important information (Dimensionality Reduction)

Detect outliers/anomalies

Find associations or similarities

Supervised

Data has inputs → labels
Output is also available

Learn label Prediction

classification, Regressions

unsupervised learning

No label → only input data

Learn patterns in data

clustering, DR, Anomaly
Detections

3) Common Techniques in Unsupervised Learning:

Clustering → grouping similar data points

Dimensionality Reduction → reducing number of features (PCA)

Anomaly Detection → finding outliers in data

4) Why use Unsupervised Learning?

To explore and understand data structure

To discover hidden patterns

To reduce complexity of data

To find anomalies or outliers

To preprocess and improve supervised models

(e.g., PCA + classifier)

5) Use Cases of Unsupervised Learning:

Customer Segmentation → group customers by behavior, spending

Market Segmentation → group markets or products

Anomaly Detection → fraud detection, network security

Recommendation Systems → find similar users/products

Image Compression → reduce image size

Medical Data → find patient groups, disease patterns

Preprocessing → use PCA to improve supervised models

Clustering

→ K means clustering

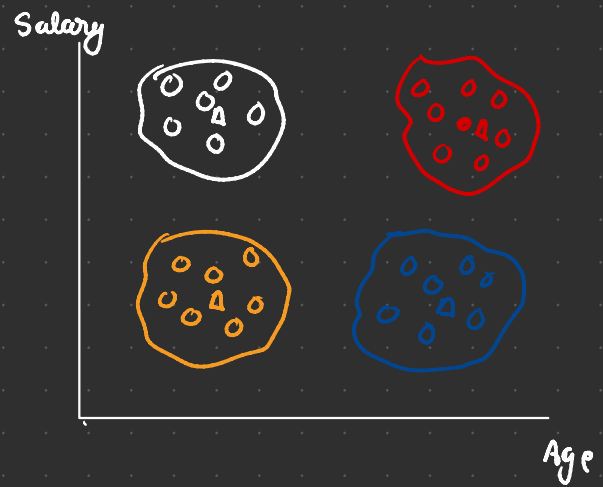
① Decide n clusters $k=4$

② initialize centroids

③ Assign cluster
↳ Euclidean Distance

④ Reassign the center point

⑤ Finest



for making the clusters and deciding k value you have to use

Elbow method

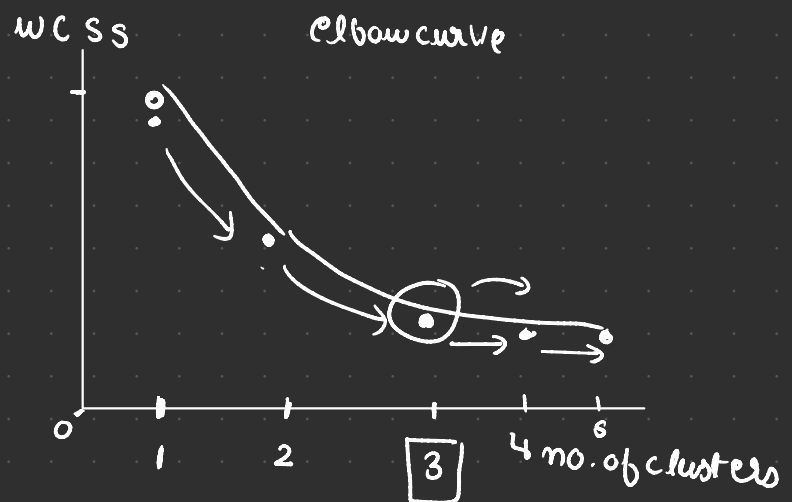
wcss → within cluster sum of square distance



$k=1$

$k=2$

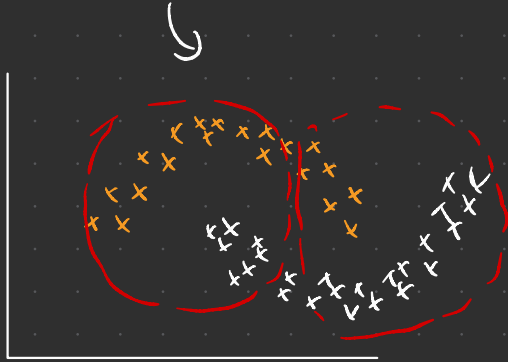
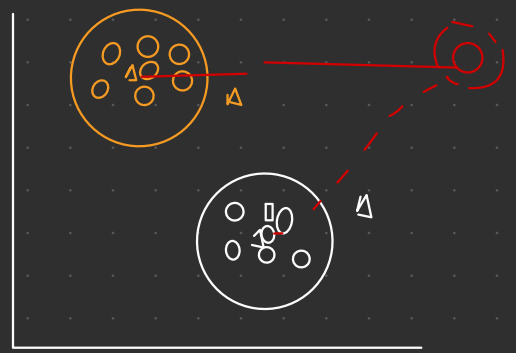
$k=3$



$$d_1^2 + d_2^2 + d_3^2 \dots d_n^2 = \text{wcss}$$

$$\text{wcss}_1 + \text{wcss}_2 + \text{wcss}_3 = \text{wcss}$$

- ① we have to initialize the k value
- ② outliers will move the center point
- ③ Non circular Data



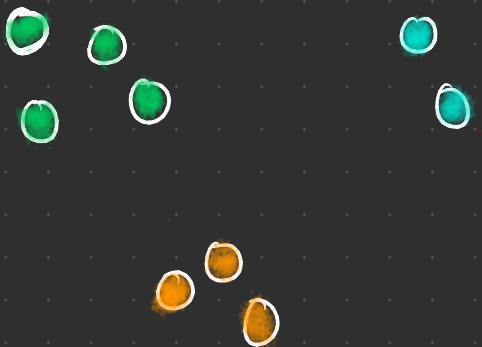
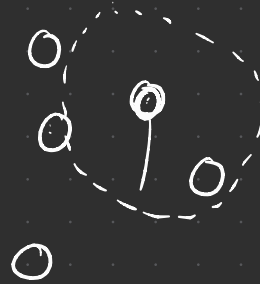
k means
↓
Centroid Based
Algo

DB scan
↓
Density Based
Algo

DB SCAN

Non Parametric Algo

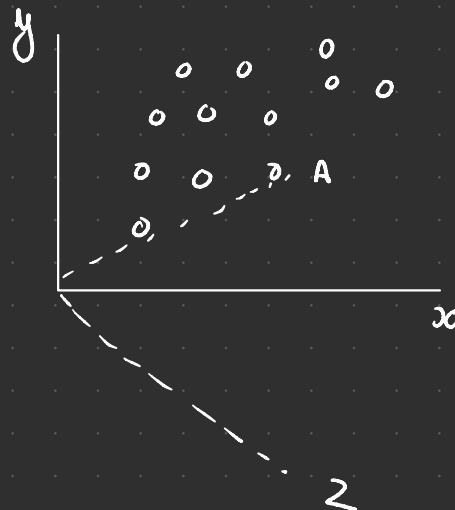
2) epsilon Distance



epsilon Distance = 1 unit

Dimensionality Reduction

1/P	2/P	3/P
x	y	z



(x, y) 2D

(x, y, z) - 3D

100D

Curse of Dimension

→ Very Difficult to find patterns

→ KNN, k-means, DBSCAN → Distance based Algo
Fail in high Dimensions

→ Storage

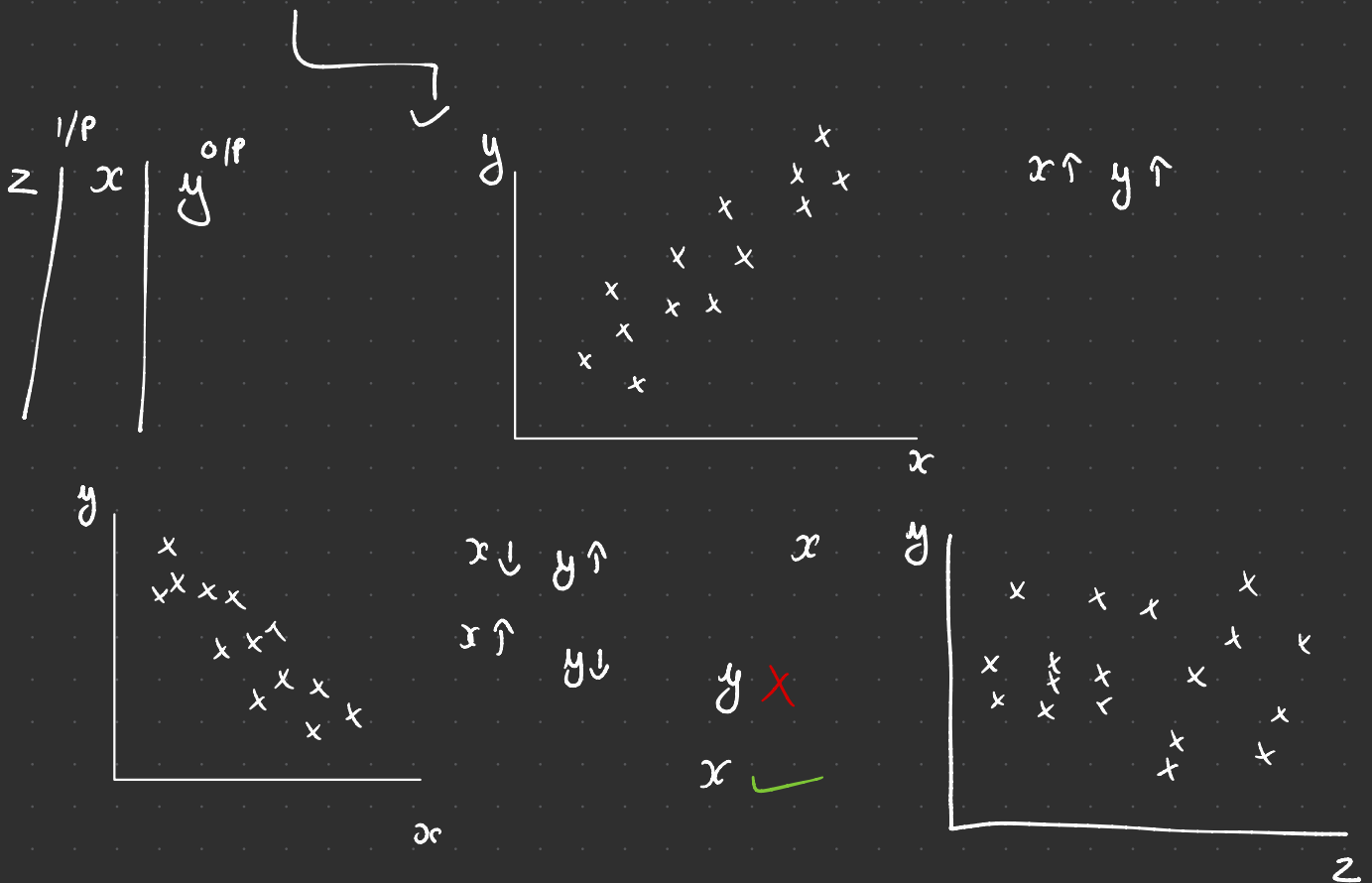
→ model training time

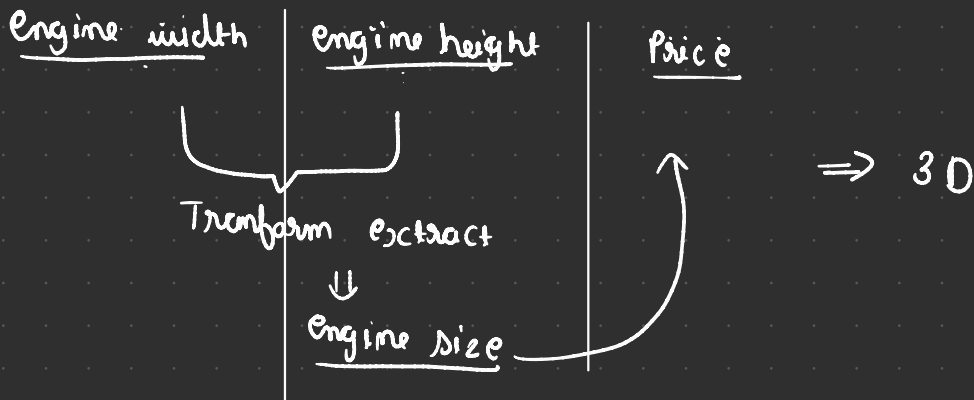
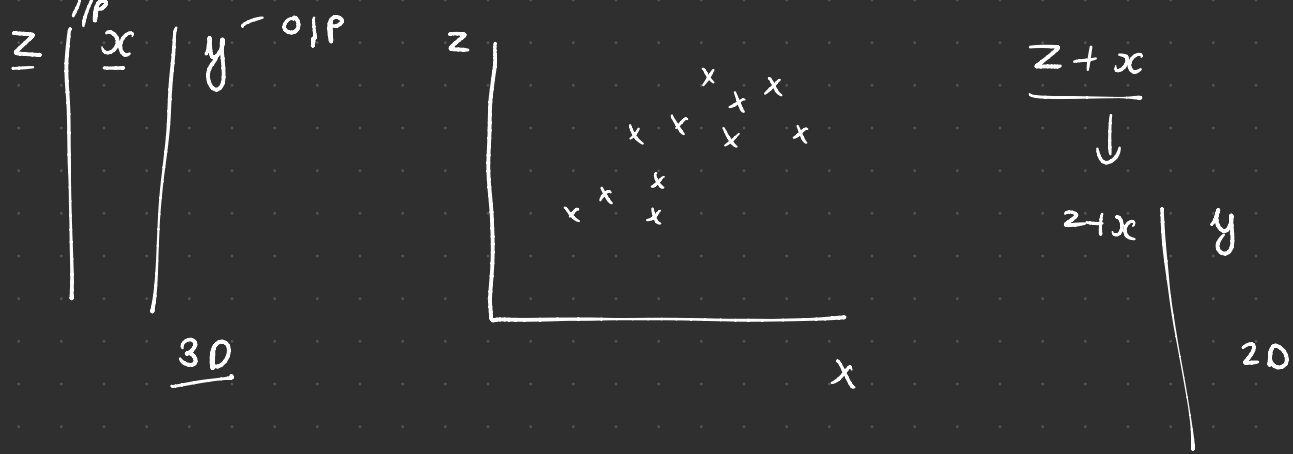
150 features → 50 features
↓ ↓
1500 → 500

how to achieve dimensionality Reduction

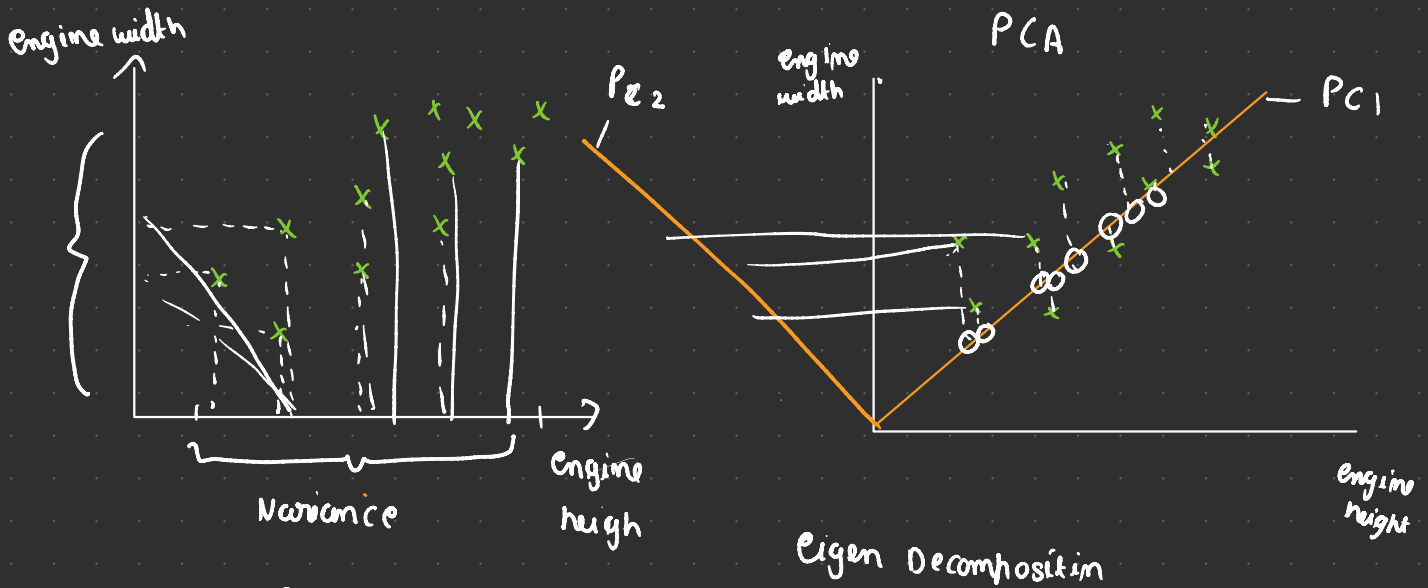
① Feature selection

② Feature extraction





Always some amount of info. will be lost but still we can predict the Price



3D - 2D

500 - 100

* To find the PCA line where maximum variance is getting captured

